

Crown & Ash

Deterministic Medieval Grand Strategy
on the Q-NarwhalKnight Blockchain



Technical Whitepaper v1.0.0

Simulation Engine v1.2.0 · ~12,000 lines Rust · ~85 tests

Q-NarwhalKnight Development Team

March 2026

Crown & Ash runs as a deterministic WASM plugin inside the
Q-NarwhalKnight quantum-enhanced DAG-BFT consensus blockchain.
Every game tick is a blockchain block. Every action is a signed transaction.
No NFTs. No token-gated gameplay. Just cryptographic truth.

Abstract

Crown & Ash is a deterministic medieval grand strategy game that executes entirely within the Q-NarwhalKnight blockchain’s WASM sandbox. The simulation manages 25 provinces across 7 factions, with full character dynasties, espionage, combat, economy, religion, and narrative generation—all using fixed-point arithmetic ($i64 \times 1000$) to guarantee identical results on every node. A 10+ phase tick pipeline processes one game turn per 10 blockchain blocks, while a 4-tier narrative cascade enriches events from raw data through template prose to optional LLM-generated dialog. The game client is built on Bevy 0.15 with ECS architecture and egui panels, connected via Server-Sent Events. This paper describes the system architecture, simulation mechanics, security model, and client design in sufficient detail to reproduce the implementation.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Design Philosophy	3
1.3	Current Status	3
2	System Architecture	3
2.1	Crate Dependency Graph	3
2.2	WASM Plugin Interface	5
2.2.1	Function ABI	5
2.2.2	Entry Points	5
2.2.3	Host Functions	5
2.3	REST & SSE API	5
3	Deterministic Foundations	6
3.1	Fixed-Point Arithmetic	6
3.2	Deterministic Random Number Generator	7
3.3	Determinism Proof Sketch	7
4	Campaign Map	8
4.1	Province Map	8
4.2	Terrain Types	8
4.3	Improvements	9
4.4	Pathfinding	9
5	The Seven Factions	9
5.1	Dynasties and Succession	10
6	Simulation Engine	10
6.1	Tick Pipeline	10
6.2	Combat System	10
6.2.1	Power Calculation	12
6.2.2	Casualty Resolution	12
6.2.3	Siege Mechanics	12
6.3	Economy	13
6.3.1	Tax Collection	13
6.3.2	Prosperity Dynamics	13
6.3.3	Population Growth	13

6.4	Realm Cohesion	13
6.5	Intrigue	14
7	NPC AI	14
7.1	Decision Priorities	14
7.2	Limitations	14
8	Narrative Engine	15
8.1	Cascade Architecture	15
8.2	Importance Classification	15
8.3	Personality Archetypes	16
8.4	LLM Generation	16
9	Client Architecture	16
9.1	Plugin Structure	16
9.2	ECS Design	16
9.3	Rendering	17
9.4	Networking	17
10	Security	17
10.1	Consensus Safety	18
10.2	Action Validation	18
10.3	Anti-Cheat	18
10.4	Work Caps	18
11	Roadmap	20
12	Conclusion	20
A	Game Constants Reference	20
B	Province Adjacency Table	21
C	Character Trait Reference	21
D	Test Coverage	22

1 Introduction

1.1 Motivation

Blockchain gaming has historically suffered from two failure modes: games that are merely “DeFi with sprites,” where the core loop is financial speculation rather than strategic depth; and games that store only asset ownership on-chain while running opaque game logic on centralized servers. Crown & Ash rejects both approaches.

The game is a *fully deterministic simulation* compiled to WebAssembly and executed inside Q-NarwhalKnight’s WASM sandbox. Every game tick corresponds to a fixed number of blockchain blocks. Every player action is a signed transaction. Every outcome—combat, economy, intrigue, succession—can be independently verified by any node replaying the block history. There are no NFTs, no token-gated mechanics, and no off-chain oracles. The blockchain *is* the game server.

1.2 Design Philosophy

1. **Determinism above all.** No floating-point arithmetic, no OS randomness, no non-deterministic iteration orders. The same block hash always produces the same game state.
2. **Strategic depth over graphical spectacle.** Crown & Ash draws inspiration from *Crusader Kings* and *Europa Universalis*: the game is won through dynasty management, diplomacy, and intrigue, not click speed.
3. **Transparency, not trust.** Any participant can verify every historical game state by replaying the chain. No hidden server-side rolls.
4. **Work-bounded execution.** Every tick has hard caps on battles, events, births, and deaths to guarantee bounded WASM execution time.

1.3 Current Status

As of simulation version 1.2.0, Crown & Ash implements:

- 25 provinces, 7 factions, 8 terrain types
- Full character system with 22+ traits, 10 personality archetypes, dynasties
- 10+ phase tick pipeline: aging, births, combat, siege, economy, trade, religion, intrigue, diplomacy, succession
- 4-tier narrative cascade (data → template → LLM dialog → deep narrative)
- WASM plugin with 4 entry points and 9 host functions
- Bevy 0.15 game client with hex map and egui panels
- REST + SSE API for real-time state streaming

2 System Architecture

2.1 Crate Dependency Graph

Crown & Ash is organized as 6 Rust crates within the Q-NarwhalKnight workspace. Figure 1 shows the dependency structure.

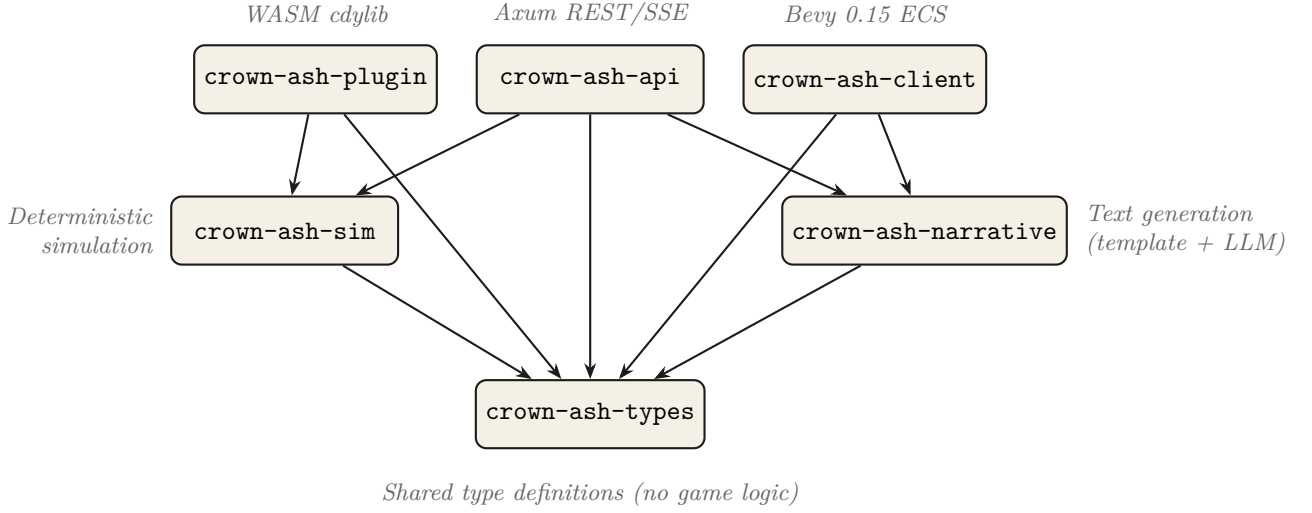


Figure 1: Crown & Ash crate dependency graph. Arrows point from dependent to dependency.

Table 1: Crate responsibilities.

Crate	Type	Purpose
crown-ash-types	lib	Shared types: <code>FixedPoint</code> , provinces, factions, characters, actions, events, world state. No game logic.
crown-ash-sim	lib	Deterministic simulation engine: tick pipeline, combat, economy, cohesion, intrigue, diplomacy, religion, trade, map, world generation.
crown-ash-narrative	lib	4-tier narrative cascade: template prose, personality archetypes, LLM prompt generation.
crown-ash-api	lib	Axum HTTP handlers: REST endpoints, SSE event streaming, game state management.
crown-ash-plugin	cdylib	WASM plugin: <code>on_init</code> , <code>on_tick</code> , <code>process_action</code> , <code>query_state</code> . Compiled to <code>.wasm</code> for on-chain execution.
crown-ash-client	bin	Bevy 0.15 game client: hex map rendering, egui UI panels, SSE networking. Excluded from workspace (bevy_egui conflicts).

2.2 WASM Plugin Interface

The simulation is compiled as a `cdylib` WASM module loaded by the Q-NarwhalKnight VM. The host-guest interface uses a simple pointer-passing ABI.

2.2.1 Function ABI

Every entry point follows the same C-ABI convention:

```

1 extern "C" fn entry(
2     input_ptr: u32,    // pointer to JSON input
3     input_len: u32,    // byte length of input
4     out_len_ptr: u32,  // write output length here
5 ) -> u32              // pointer to JSON output

```

Listing 1: WASM entry point signature.

The host serializes input as JSON into the guest's linear memory via a bump allocator starting at offset 65536 (above the WASM stack). The guest deserializes, executes, serializes the response, and returns a pointer to it.

2.2.2 Entry Points

Table 2: WASM plugin entry points.

Entry Point	Input	Output	Purpose
<code>on_init</code>	<code>InitRequest</code>	<code>InitResponse</code>	Create world from config + seed
<code>on_tick</code>	<code>TickRequest</code>	<code>TickResponse</code>	Advance one game turn
<code>process_action</code>	<code>ActionRequest</code>	<code>ActionResponse</code>	Execute a player action
<code>query_state</code>	<code>QueryRequest</code>	<code>QueryResponse</code>	Read-only state query

2.2.3 Host Functions

The WASM guest imports 9 host functions for storage, events, and utilities:

Table 3: Host functions imported by the WASM plugin.

Host Function	Purpose
<code>plugin_storage_read</code>	Read a value from persistent plugin storage
<code>plugin_storage_write</code>	Write a value to persistent plugin storage
<code>plugin_storage_delete</code>	Delete a key from plugin storage
<code>plugin_storage_exists</code>	Check if a key exists
<code>plugin_emit_event</code>	Emit a typed blockchain event (topic + data)
<code>plugin_get_block_height</code>	Current block height (u64)
<code>plugin_get_timestamp</code>	Current UNIX timestamp (u64)
<code>plugin_sha3_256</code>	Compute SHA3-256 hash of arbitrary data
<code>plugin_log</code>	Log message at levels 0 (trace) through 4 (error)

2.3 REST & SSE API

The `crown-ash-api` crate exposes game state through Axum HTTP handlers:

Real-time updates are delivered via Server-Sent Events (SSE) on the main Q-NarwhalKnight SSE stream. Four event types carry narrative content:

Table 4: Crown & Ash REST API endpoints.

Method	Path	Description
GET	/world	Full world state snapshot
GET	/province/:id	Single province details
GET	/character/:id	Single character details
GET	/faction/:id	Faction details + relations
POST	/action	Queue a signed player action
POST	/join	Player joins a game session

- `crown_ash_event` — Tier 0 structured JSON data
- `crown_ash_prose` — Tier 1 template-generated text
- `crown_ash_dialog` — Tier 2 LLM-generated character dialog
- `crown_ash_epic` — Tier 3 deep LLM narrative

3 Deterministic Foundations

Every node in the Q-NarwhalKnight network must compute identical game states. This section describes the two mechanisms that guarantee determinism: fixed-point arithmetic and a seeded pseudo-random number generator.

3.1 Fixed-Point Arithmetic

Crown & Ash bans IEEE 754 floating-point entirely. All numeric computation uses a custom `FixedPoint` type:

```

1 pub struct FixedPoint(pub i64);
2
3 const SCALE: i64 = 1000; // 3 decimal places
4
5 impl FixedPoint {
6     pub fn from_int(n: i64) -> Self { Self(n * SCALE) }
7     pub fn from_fraction(num: i64, den: i64) -> Self {
8         Self(num * SCALE / den)
9     }
10    pub fn mul_fp(self, rhs: Self) -> Self {
11        Self(self.0 * rhs.0 / SCALE)
12    }
13    pub fn checked_div_fp(self, rhs: Self) -> Option<Self> {
14        if rhs.0 == 0 { None }
15        else { Some(Self(self.0 * SCALE / rhs.0)) }
16    }
17 }
```

Listing 2: FixedPoint core definition.

Key properties:

- **Scale factor:** $1000 \times (10^3)$. The value `FixedPoint(2750)` represents 2.750.
- **Range:** $[-9.22 \times 10^{15}, 9.22 \times 10^{15}]$, sufficient for all game quantities.

- **Overflow safety:** Multiplication uses i64 intermediate products. For the value ranges in Crown & Ash (populations $\leq 10^5$, gold $\leq 10^6$), overflow is impossible.
- **Division by zero:** `checked_div_fp` returns `None`; `saturating_div_fp` returns `ZERO`.
- **Linear interpolation:** `lerp(other, t)` where $t \in [0, 1000]$ blends between two values without floating-point.

3.2 Deterministic Random Number Generator

Randomness is derived exclusively from block hashes using an ARX (add-rotate-xor) hash mixer:

```

1 pub struct DeterministicRng {
2     state: [u8; 32],
3 }
4
5 impl DeterministicRng {
6     pub fn new(block_hash: [u8; 32], domain: &str) -> Self {
7         // state = hash(block_hash || domain || "CROWN_ASH")
8         let mut hasher = ArxHash::new();
9         hasher.update(&block_hash);
10        hasher.update(domain.as_bytes());
11        hasher.update(b"CROWN_ASH");
12        Self { state: hasher.finalize() }
13    }
14 }
```

Listing 3: Deterministic RNG construction.

Domain separation ensures that different subsystems (combat, economy, events) receive independent pseudo-random sequences from the same block hash. For example, `DeterministicRng::new(hash, "combat")` and `DeterministicRng::new(hash, "economy")` produce unrelated streams.

The ARX mixing function applies 16 rounds of add-rotate-xor over 8 state words initialized from the NIST π fractional digits. This is a non-cryptographic hash optimized for speed and avalanche properties, not collision resistance.

Output methods:

- `next_u32()` — Extract 4 bytes from state as little-endian u32, then re-hash.
- `range(min, max)` — Uniform i64 in $[\text{min}, \text{max})$.
- `chance(num, den)` — Returns `true` with probability $\frac{\text{num}}{\text{den}}$.

3.3 Determinism Proof Sketch

1. **Inputs are deterministic.** Block hashes are consensus-agreed. Player actions are signed transactions ordered by DAG-Knight consensus.
2. **Arithmetic is deterministic.** `FixedPoint` uses only i64 operations with defined overflow semantics.
3. **RNG is deterministic.** Same block hash + same domain $\Rightarrow$ same sequence.
4. **Iteration order is deterministic.** All collections are `Vec` (ordered) or sorted by key before iteration. No `HashMap` iteration in game logic.
5. **No external I/O.** The WASM sandbox prohibits file, network, and clock access. `plugin_get_timestamp` returns the consensus-agreed block timestamp.

Therefore, for any block sequence $B_1, B_2, \dots, B_n$, all nodes compute identical game states $S_0 \xrightarrow{B_1} S_1 \xrightarrow{B_2} S_2 \dots \xrightarrow{B_n} S_n$.

4 Campaign Map

The campaign map consists of 25 provinces organized into 7 regional clusters, each initially controlled by one faction. Province adjacency forms a connected graph with approximately 63 undirected edges.

4.1 Province Map

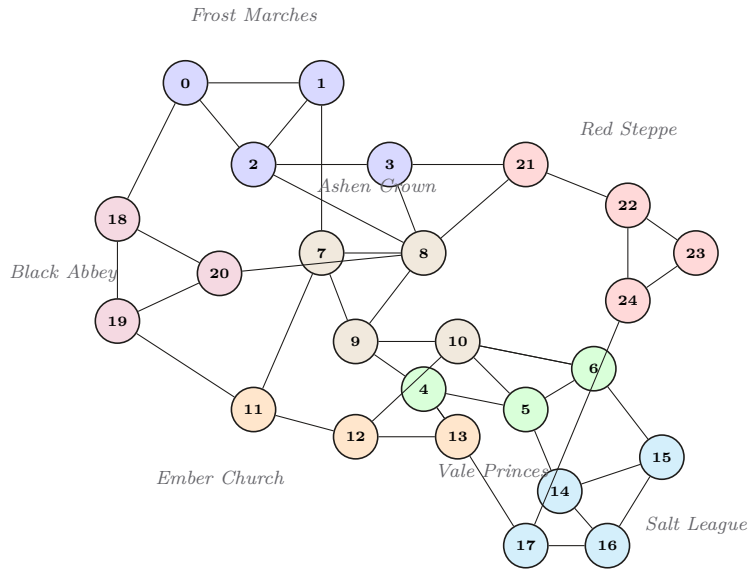


Figure 2: Campaign map: 25 provinces across 7 factions. Numbers are province IDs; colors indicate starting controller.

4.2 Terrain Types

Each province has an assigned terrain type that affects movement cost and defensive bonuses in combat.

Table 5: Terrain types with movement costs and defense bonuses.

Terrain	Movement Cost	Defense Bonus
Plains	1.000	+0%
Hills	1.500	+20%
Mountains	2.000	+40%
Forest	1.300	+25%
Marsh	1.800	+30%
Desert	1.400	+10%
Coastal	1.000	+15%
River	1.200	+20%

4.3 Improvements

Players can construct improvements in their provinces. Each has a gold cost, build time (in turns), and economic or military output.

Table 6: Province improvements.

Improvement	Cost	Turns	Effect
Farmstead	50	3	+10 food/turn
Mine	80	5	+5 iron, +3 gold/turn
Lumbercamp	40	3	+8 timber/turn
Quarry	60	5	+6 stone/turn
Stables	70	5	+5 horses/turn
Market	100	7	+2.000 prosperity, +gold
Temple	120	7	+clerical favor
Fortification	150	12	+defense level (+10%/level)
University	200	10	+learning bonus
Port (coastal)	130	10	+trade capacity
Granary	45	3	Famine resistance
Hospital	90	7	Plague resistance

4.4 Pathfinding

Province connectivity is stored as an adjacency list. Pathfinding uses breadth-first search, which is optimal for unweighted graphs and trivially bounded: $O(V + E) = O(25 + 63) = O(88)$.

```

1 pub fn find_path(start: u16, goal: u16) -> Option<Vec<u16>>;
2 pub fn path_length(start: u16, goal: u16) -> Option<usize>;

```

Listing 4: BFS pathfinding API.

5 The Seven Factions

Each faction has a unique culture, religion, starting provinces, and a set of `FixedPoint` multipliers that scale military, economic, diplomatic, and religious effectiveness.

Table 7: Faction bonuses and characteristics.

ID	Name	Culture	Religion	Mil.	Econ.	Dipl.	Decay
0	Ashen Crown	Imperial	Ember Church	0.900	1.000	1.200	0.900
1	Vale Princes	Feudal	Ember Church	1.100	1.300	1.000	1.200
2	Ember Church	Clerical	Ember Church	0.800	1.000	1.000	0.800
3	Salt League	Mercantile	Salt Cult	1.000	1.500	1.200	1.100
4	Frost Marches	Nordic	Frost Spirits	1.400	1.000	0.800	1.000
5	Red Steppe	Nomadic	Old Faith	1.300	0.600	1.000	1.300
6	Black Abbey	Monastic	Black Order	1.000	1.000	0.800	1.000

Decay is the cohesion decay rate multiplier (§6.4). A value below 1.000 means the faction’s realm stability decays slower than baseline (institutional advantage); above 1.000 means faster decay (nomadic instability).

5.1 Dynasties and Succession

Each faction’s ruling dynasty is tracked through a full character graph including marriages, children, and inheritance claims. When a ruler dies, succession follows these rules:

1. **Designated heir:** If the ruler used `DesignateHeir`, that character inherits.
2. **Eldest child:** The oldest living child of the same dynasty.
3. **Dynasty member:** Any living character of the same dynasty.
4. **Succession crisis:** If no valid heir exists, the realm may fracture—triggering a `RealmSplit` event.

A succession crisis can cause provinces to defect to a rebel faction led by the strongest rival claimant, reducing the original realm’s territory.

6 Simulation Engine

The simulation engine (`crown-ash-sim`) is the heart of Crown & Ash. It processes one game turn per invocation of `tick()`, which is called every `BLOCKS_PER_TURN = 10` blockchain blocks.

6.1 Tick Pipeline

Figure 3 shows the 10+ phase pipeline executed each turn.

Algorithm 1 Tick Pipeline (simplified)

Require: *world*: mutable GameWorld, *hash*: 32-byte block hash

Ensure: *summary*: TurnSummary (list of events)

- 1: *rng* $\leftarrow$ DETERMINISTICRNG(*hash*)
 - 2: *events* $\leftarrow$ []
 - 3: AGECHARACTERS(*world*, *rng*, *events*) ▷ natural death
 - 4: PROCESSBIRTHS(*world*, *rng*, *events*) ▷ cap: 5/turn
 - 5: EDUCATION(*world*, *rng*)
 - 6: PROCESSACTIONS(*world*, *rng*, *events*) ▷ player + NPC AI
 - 7: RESOLVEBATTLES(*world*, *rng*, *events*) ▷ cap: 5/turn
 - 8: PROCESSSIEGES(*world*, *rng*, *events*)
 - 9: MOVEARMIES(*world*)
 - 10: ECONOMY(*world*, *events*)
 - 11: TRADE(*world*, *events*)
 - 12: COHESION(*world*) ; UNREST(*world*)
 - 13: RELIGION(*world*, *rng*, *events*)
 - 14: INTRIGUE(*world*, *rng*, *events*) ▷ cap: 20 plots
 - 15: DIPLOMACY(*world*, *events*)
 - 16: RANDEVENTS(*world*, *rng*, *events*) ▷ cap: 10/turn
 - 17: SUCCESSIONCHECK(*world*, *rng*, *events*) ▷ cap: 3/turn
 - 18: LIFE_CYCLE(*world*) ▷ tombstone dead, enforce caps
 - 19: **return** TURN_SUMMARY(*events*)
-

6.2 Combat System

Battles are resolved automatically when opposing armies occupy the same province.

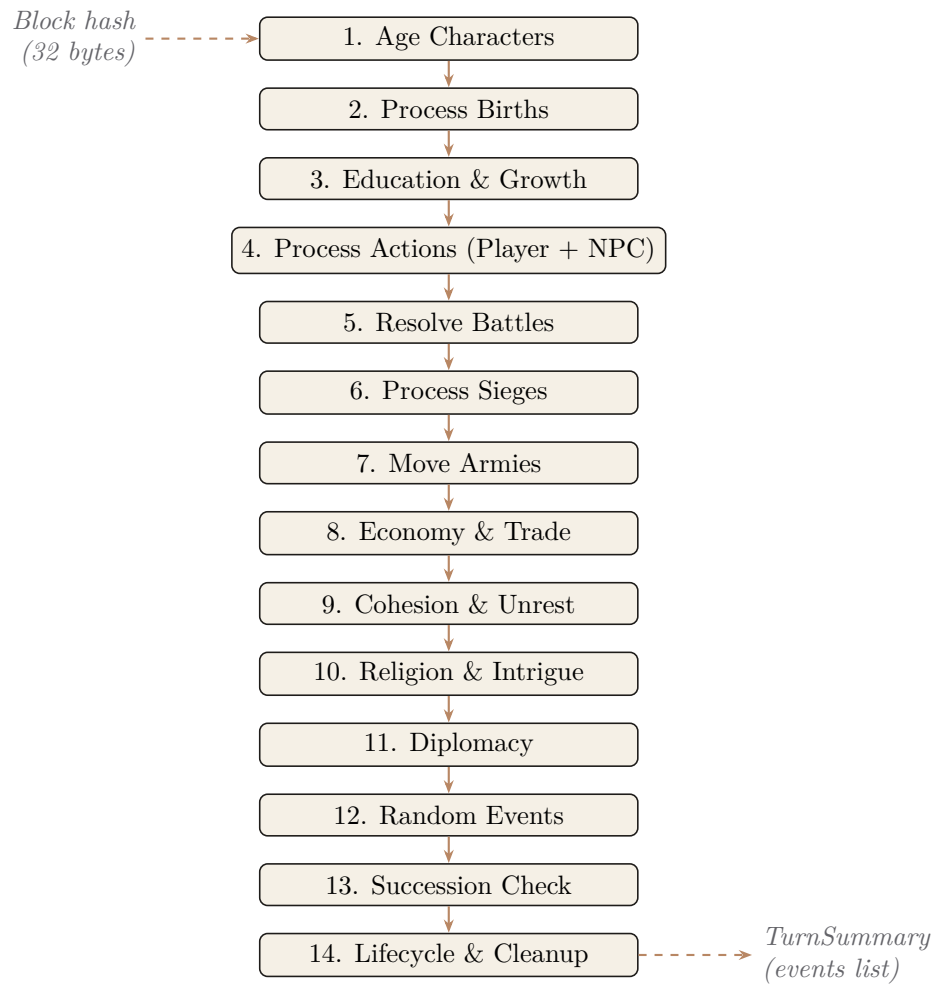


Figure 3: Tick pipeline: 14 phases per game turn, seeded by the block hash.

6.2.1 Power Calculation

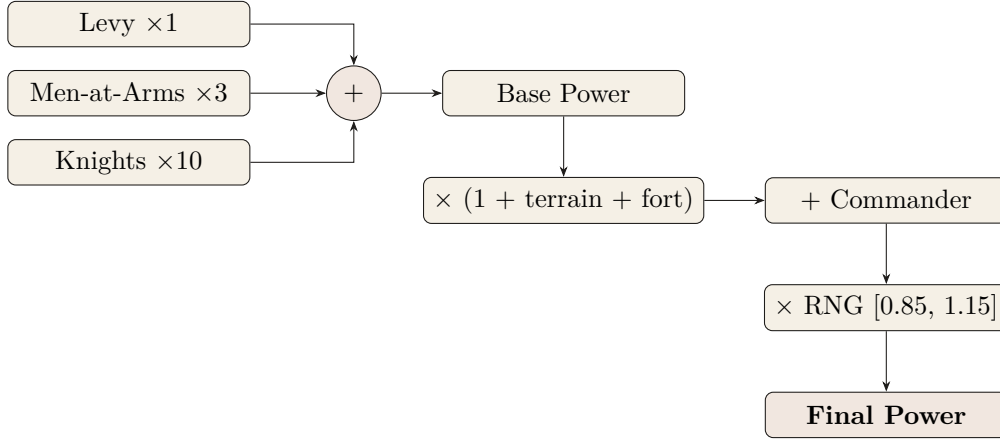


Figure 4: Combat power calculation flow.

The combat formula in `FixedPoint` arithmetic:

$$\text{attack} = \text{levy} \times 1 + \text{men_at_arms} \times 3 + \text{knight} \times 10 \quad (1)$$

$$\text{defense} = \text{attack} \times \frac{1000 + \text{terrain_bonus} + \text{fort_bonus}}{1000} \quad (2)$$

$$\text{cmd_bonus} = \text{martial_stat} \times 10 \quad (3)$$

$$\text{rng_factor} \sim \text{Uniform}[850, 1150] \quad (4)$$

$$\text{final} = (\text{power} + \text{cmd_bonus}) \times \frac{\text{rng_factor}}{1000} \quad (5)$$

6.2.2 Casualty Resolution

- **Winner casualties:** 20–30% of engaged troops (base 200–300).
- **Loser casualties:** 40–60% of engaged troops (base 400–600).
- **Morale collapse:** If casualties exceed 40% of the original army size, morale drops to 100 (minimum), preventing further offensive action until reinforced.
- **Retreat:** The losing army retreats to a random adjacent province controlled by their faction. Armies with no valid retreat destination are destroyed.

6.2.3 Siege Mechanics

Sieges occur when an army attacks a fortified province. Unlike field battles, sieges resolve over multiple turns:

- **Duration:** Proportional to fortification level (base 3 turns + 2/level).
- **Attrition:** Besieging army takes casualties each turn.
- **Completion:** Province control transfers on siege completion.

6.3 Economy

6.3.1 Tax Collection

Each turn, provinces generate tax revenue:

$$\text{tax_gold} = \frac{\text{population} \times \text{tax_rate} \times \text{prosperity}}{1,000,000} \quad (6)$$

where all values are `FixedPoint` (scale 1000). Tax rate is player-configurable from 0 to 1000 (0% to 100%).

6.3.2 Prosperity Dynamics

Prosperity is the primary economic health indicator, ranging from 50.000 (destitute) to 1000.000 (flourishing):

Table 8: Prosperity modifiers per turn.

Condition	Change	Notes
Peace (no war, unrest ≤ 500)	+2.000	Baseline growth
Active war	−5.000	Faction at war
Farmstead improvement	+1.000	Per turn bonus
Market improvement	+2.000	Per turn bonus

6.3.3 Population Growth

- **Growth:** If prosperity > 500 , population grows by +0.1% per turn.
- **Decline:** If prosperity < 200 , population declines by −0.2% per turn.
- **Floor:** No province drops below 100 population.

6.4 Realm Cohesion

Realm stability is modeled through five independent cohesion components, each ranging from 0 to 1000:

Table 9: Cohesion components.

Component	Description
Legitimacy	The ruler’s perceived right to rule. Boosted by high martial stat.
Fealty	Vassal loyalty. Boosted by high diplomacy.
Clerical Favor	Church/religious institution support. Boosted by high learning.
Commoner Mood	General population happiness. Reduced by war, boosted by stewardship.
Regional Identity	Cultural unity. Drops sharply (−30) when a province is conquered.

Each component decays toward equilibrium (500) at a base rate of 0.010/turn, modified by the faction’s decay multiplier. If *any* component drops below 200, the realm risks fracture: provinces may rebel, triggering a `RealmSplit` event.

6.5 Intrigue

The intrigue system models espionage through four plot types:

Table 10: Intrigue plot types and effects.

Plot Type	Effect	Fire Threshold	Notes
Assassination	Kill target character	1000	70% base success
Fabricate	Create province claim	1000	Enables war
Sabotage	−200 prosperity	1000	Economic warfare
Steal	−100 gold from treasury	1000	Subterfuge

Plot progress advances each turn:

$$\text{progress} = \text{instigator.intrigue} \times 20 + \sum_{\text{backers}} \text{backer.intrigue} \times 5 \quad (7)$$

Plots fire when progress reaches 1000. **Counter-intelligence** is handled by the target’s spymaster: if $\text{spymaster.intrigue} \times 10 > \text{plot.secretcy}$, the plot is discovered. The Paranoid trait grants +200 discovery bonus.

Work cap: Maximum 20 active plots globally, 3 executions per turn.

7 NPC AI

Non-player factions are controlled by a heuristic decision framework that runs during phase 4 (Process Actions) of each tick. The AI evaluates the faction’s current state and selects actions from the same `GameAction` enum available to human players.

7.1 Decision Priorities

The AI evaluates actions in priority order:

1. **Survival:** If any cohesion component < 250 , prioritize stabilization (lower taxes, appease clergy, assign steward).
2. **Military threat:** If enemy armies are in controlled provinces, raise armies and engage.
3. **Expansion:** If military strength exceeds neighbors by $> 50\%$ and legitimacy > 600 , consider declaring war.
4. **Economy:** Build improvements in provinces with the highest ROI (markets in high-population provinces, farmsteads in low-prosperity).
5. **Intrigue:** Rulers with the Schemer or Deceitful trait are more likely to launch plots against rivals.
6. **Diplomacy:** Propose treaties with factions that share a religion or have high mutual opinion.

7.2 Limitations

The current NPC AI is deliberately simple:

- No multi-turn planning or look-ahead.

- No coalition formation.
- Personality traits influence action *weights* but do not fundamentally change the decision tree.
- The AI uses the same deterministic RNG as all other systems, ensuring reproducibility.

[Advanced AI behaviors (feudal obligation simulation, bluffing in diplomacy, strategic retreat) are planned for future versions. — to be expanded in future versions.]

8 Narrative Engine

Crown & Ash transforms raw game events into prose through a 4-tier narrative cascade. The system is designed to provide instant feedback for routine events while reserving expensive LLM generation for dramatic moments.

8.1 Cascade Architecture

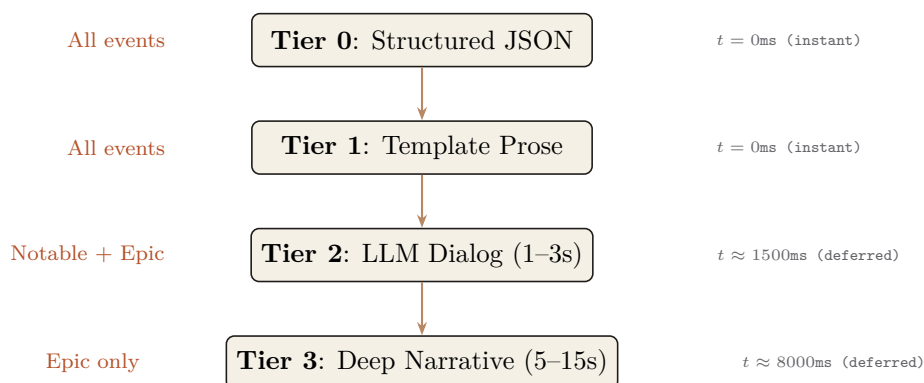


Figure 5: 4-tier narrative cascade. Lower tiers fire only for high-importance events.

8.2 Importance Classification

Events are classified into three importance levels that determine which narrative tiers activate:

Table 11: Event importance classification.

Level	Trigger Examples	Tiers
Minor	Harvest, construction complete, routine diplomacy	0, 1
Notable	Province conquered, war declared, treaty signed, plot success, rebellion (> 200 rebels)	0, 1, 2
Epic	Faction eliminated, realm split, succession crisis, massive battle (> 500 casualties)	0, 1, 2, 3

8.3 Personality Archetypes

Character dialog is flavored by personality archetypes derived from trait combinations. The **crown-ash-narrative** crate maps each character’s traits to one of 10 archetypes, which provides tone descriptors and speech patterns for LLM prompting.

Table 12: Personality archetypes and their derivation.

Archetype	Traits Required	Tone & Speech
Tyrant	Cruel/Wrathful + Brave	Menacing, authoritative. “Obedience or death.”
Saint	Pious + Kind/Just	Gentle, compassionate. “May the gods guide us.”
Schemer	Schemer / Deceitful+Ambitious	Calculating, smooth. “Trust is a luxury.”
Commander	Brave + Just/Strategist	Decisive, honorable. “For glory!”
Diplomat	Gregarious + Kind/Honest	Warm, persuasive. “Peace through trade.”
Scholar	Scholar / Patient+Diligent	Measured, thoughtful. “Knowledge is power.”
Conqueror	Ambitious + Wrathful/Brave	Bold, commanding. “Seize destiny!”
Hedonist	Content + Slothful/Gluttonous	Languid, amused. “Enjoy the feast.”
Inquisitor	Paranoid + Cruel/Cynical	Suspicious, sharp. “Who can be trusted?”
Balanced	No dominant pattern	Measured, pragmatic. Neutral tone.

8.4 LLM Generation

Tier 2 and Tier 3 narratives are generated by passing structured context to an LLM:

- **Tier 2 (dialog):** Short character speech (1–3 sentences). The prompt includes the character’s archetype tone, the event context, and faction relationships. Generated asynchronously; delivered via **crown_ash_dialog** SSE events.
- **Tier 3 (deep narrative):** Multi-paragraph dramatic prose describing epic events from a narrator’s perspective. Only triggered for Epic-class events. Delivered via **crown_ash_epic** SSE events.

The LLM is accessed through the Q-NarwhalKnight node’s inference engine (when available). If no LLM is available, only Tier 0 and Tier 1 output is generated—the game remains fully playable without narrative enrichment.

9 Client Architecture

The Crown & Ash client is built on Bevy 0.15 with an Entity Component System (ECS) architecture. It is compiled as a standalone binary, separate from the workspace (due to bevy_egui dependency conflicts).

9.1 Plugin Structure

9.2 ECS Design

- **Provinces:** One entity per province with terrain mesh components, colored by controlling faction.

Table 13: Bevy client plugins.

Plugin	Systems	Resources
CrownAshNetworkPlugin	API polling, SSE listener	API state, SSE stream
CrownAshMapPlugin	Map rendering, terrain display	Map state, province meshes
CrownAshUiPlugin	UI panels, action submission, narrative display	Game state, narrative state, selection state

- **Armies:** Entities with position, troop count, and faction marker components.
- **Selection:** A global `SelectionState` resource tracks the currently selected province, army, or character for UI panel display.
- **Narrative:** A `NarrativeState` resource queues incoming SSE events and tier-based narrative text for the event log panel.

9.3 Rendering

The map is rendered as a 2D hex grid using Bevy’s sprite system:

- Province territories are colored by faction ownership.
- Terrain types use distinct visual patterns.
- Armies are represented as shield icons with troop count labels.
- UI panels use egui for immediate-mode rendering of faction info, province details, character sheets, and the event log.

9.4 Networking

The client connects to the Q-NarwhalKnight node via:

1. **REST polling:** Fetches the full `GameWorld` on startup and periodically refreshes.
2. **SSE streaming:** Subscribes to the node’s SSE endpoint for real-time `crown_ash_*` events, eliminating the need for full-state polling after initial sync.
3. **Action submission:** Player actions are POSTed to `/action` as signed JSON payloads.

10 Security

Running a strategy game on a public blockchain introduces unique security considerations. Crown & Ash addresses these through multiple layers of defense.

10.1 Consensus Safety

Because the game state is computed deterministically from the block sequence, consensus safety inherits directly from Q-NarwhalKnight's DAG-BFT protocol:

- **Finality:** Game state changes are final once the corresponding blocks achieve DAG-Knight finality ($< 2.9s$).
- **Fork resistance:** If a chain fork occurs, both branches produce valid but divergent game states. On fork resolution, the losing branch's game state is discarded and recomputed from the winning chain.
- **SIM_VERSION check:** Nodes with mismatched simulation versions reject each other's game state, preventing desynchronization.

10.2 Action Validation

Player actions undergo multi-layer validation:

1. **Signature verification:** Every action is a signed transaction. Invalid signatures are rejected before processing.
2. **Wallet-faction binding:** A player's wallet is bound to exactly one faction at join time. Actions targeting provinces or characters outside the player's faction are rejected.
3. **Rule validation:** The simulation engine validates action legality (e.g., cannot raise armies in uncontrolled provinces, cannot declare war on allies, cannot build improvements already under construction).
4. **Rate limiting:** One action per player per turn, enforced at the API layer.

10.3 Anti-Cheat

- **No client authority:** The client is a pure display layer. All state transitions occur in the WASM sandbox. A modified client cannot produce invalid state.
- **Deterministic verification:** Any observer can replay the chain and verify every game state transition. Cheating would require compromising blockchain consensus itself.
- **No hidden information:** All game state is on-chain and visible to all nodes. There is no fog of war in the current version. *[Encrypted state with zero-knowledge proofs for fog-of-war. — to be expanded in future versions.]*

10.4 Work Caps

To prevent denial-of-service through computationally expensive game states, every tick phase has hard limits:

These caps ensure that WASM execution time remains bounded regardless of game state complexity, preventing a single game from starving the blockchain of resources.

Table 14: Per-turn work caps.

Operation	Max per Turn
Battles resolved	5
Random events	10
Births	5
Deaths	5
Succession checks	3
Active plots (global)	20
Plot executions	3

Table 15: Development roadmap. Update the **Status** column as work progresses.

Phase	Name	Features	Status
1	Core Simulation	Tick pipeline, combat, economy, factions, provinces, map	COMPLETE
2	Character Depth	Traits, dynasties, succession, marriage, education	COMPLETE
3	Espionage	Intrigue plots, counter-intelligence, spymasters	COMPLETE
4	Narrative	4-tier cascade, personality archetypes, LLM integration	COMPLETE
5	Client	Bevy 0.15 client, hex map, egui panels, SSE networking	COMPLETE
6	Diplomacy+	Advanced treaties, coalition wars, tributary system	IN PROGRESS
7	Religion+	Great schisms, heresies, crusade mechanics	PLANNED
8	Fog of War	ZK-encrypted state, selective revelation	PLANNED
9	Modding	User-uploadable WASM scenario packs	PLANNED
10	Multiplayer+	Competitive seasons, ranked matches, spectator mode	PLANNED

11 Roadmap

12 Conclusion

Crown & Ash demonstrates that complex, deep strategy games can run *entirely* on a blockchain without sacrificing gameplay depth for financial mechanics. By compiling the simulation to deterministic WASM, using fixed-point arithmetic throughout, and seeding randomness from consensus-agreed block hashes, every node independently computes identical game states—making the blockchain itself the authoritative game server.

The 6-crate architecture separates concerns cleanly: types define the data model, the simulation engine processes game logic, the narrative engine adds literary flavor, the plugin compiles to on-chain WASM, the API exposes state to clients, and the Bevy client renders it all. This modularity enables independent development of each layer and straightforward extension as new game systems (advanced diplomacy, fog of war, modding) are added.

Crown & Ash is not finished—it is a living system designed for continuous expansion. The version macros, modular sections, and roadmap table in this paper are structured for easy updates as the game evolves.

A Game Constants Reference

Table 16: Complete game constants reference.

Constant	Module	Value	Description
SIM_VERSION	world	1.2.0	Simulation version string
BLOCKS_PER_TURN	world	10	Blocks per game turn
MAX_FACTIONS	world	7	Maximum factions
MAX_PROVINCES	world	25	Maximum provinces
SCALE	fixed_point	1000	FixedPoint scale factor
MAX_BATTLES_PER_TURN	combat	5	Battle resolution cap
MAX_EVENTS_PER_TURN	tick	10	Random event cap
MAX_SUCCESSIONS_PER_TURN	tick	3	Succession check cap
MAX_BIRTHS_PER_TURN	tick	5	Birth event cap
MAX_DEATHS_PER_TURN	tick	5	Death event cap
BASE_DECAY_RATE	cohesion	0.010	Cohesion decay/turn
WAR_MOOD_PENALTY	cohesion	−3.000	Commoner mood per turn at war
CONQUEST_IDENTITY_PENALTY	cohesion	−30.000	One-time identity loss
FORT_BONUS_PER_LEVEL	combat	100	+10% defense per fort level
MORALE_COLLAPSE	combat	0.400	40% casualty threshold
ASSASSINATION_BASE	intrigue	700	70% base success chance
ASSASSINATION_BONUS	intrigue	30	+3% per intrigue point
MAX_ACTIVE_PLOTS	intrigue	20	Global plot cap
PLOT_FIRE_THRESHOLD	intrigue	1000	Progress to execute
SECRECY_DECAY	intrigue	10	Secrecy loss per turn
DISCOVERY_PENALTY	intrigue	100	Opinion loss on discovery
PROSPERITY_MIN	economy	50.000	Province prosperity floor
PROSPERITY_MAX	economy	1000.000	Province prosperity ceiling

Continued on next page

Constant	Module	Value	Description
POPULATION_MIN	economy	100	Province population floor
POP_GROWTH_RATE	economy	0.001	+0.1%/turn if prosperous
POP_DECLINE_RATE	economy	0.002	−0.2%/turn if destitute

B Province Adjacency Table

Table 17: Full province adjacency list.

ID	Name	Faction	Adjacent To
0	Frosthald	Frost Marches	1, 2, 18
1	Winterfell Vale	Frost Marches	0, 2, 7
2	Icemere	Frost Marches	0, 1, 3, 8
3	Stormwatch	Frost Marches	2, 8, 21
4	Goldhaven	Vale Princes	5, 9, 13
5	Thornwall	Vale Princes	4, 6, 10, 14
6	Ravensgate	Vale Princes	5, 10, 15
7	Ashenmere	Ashen Crown	1, 8, 9, 11
8	Crownspire	Ashen Crown	2, 3, 7, 9, 20, 21
9	Embervale	Ashen Crown	4, 7, 8, 10
10	Kingsreach	Ashen Crown	5, 6, 9, 12
11	Sanctum	Ember Church	7, 12, 19
12	Pyrelight	Ember Church	10, 11, 13
13	Candlekeep	Ember Church	4, 12, 17
14	Saltmere	Salt League	5, 15, 16
15	Tidehollow	Salt League	6, 14, 16
16	Coinport	Salt League	14, 15, 17
17	Warehouse Row	Salt League	13, 16, 24
18	Shadowmere	Black Abbey	0, 19, 20
19	Whispering Cloister	Black Abbey	11, 18, 20
20	Veilstone	Black Abbey	8, 18, 19
21	Khanstead	Red Steppe	3, 8, 22
22	Windbreak	Red Steppe	21, 23, 24
23	Dustmane	Red Steppe	22, 24
24	Redhorn	Red Steppe	17, 22, 23

C Character Trait Reference

Table 18: Character trait stat modifiers (FixedPoint values).

Trait	Martial	Diplomacy	Stewardship	Intrigue	Learning
Brave	+3	—	—	—	—
Strategist	+5	—	—	—	—
Craven	−3	—	—	—	—
Wrathful	+2	—	—	—	—
Just	—	+2	—	—	—

Continued on next page

Trait	Martial	Diplomacy	Stewardship	Intrigue	Learning
Kind	—	+3	—	—	—
Gregarious	—	+4	—	—	—
Cruel	—	−3	—	—	—
Shy	—	−2	—	—	—
Honest	—	+1	—	−2	—
Diligent	—	—	+3	—	—
Administrator	—	—	+5	—	—
Temperate	—	—	+2	—	—
Gluttonous	—	—	−2	—	—
Slothful	—	—	−3	—	—
Schemer	—	—	—	+5	—
Deceitful	—	—	—	+3	—
Paranoid	—	—	—	+2	—
Trusting	—	—	—	−3	—
Scholar	—	—	—	—	+5
Theologian	—	—	—	—	+3
Patient	—	—	—	—	+2

D Test Coverage

The following test suites validate Crown & Ash’s simulation logic:

Total: ~85 tests across all modules. Run with:

```
cargo test --package crown-ash-sim
cargo test --package crown-ash-types
cargo test --package crown-ash-narrative
cargo test --package crown-ash-plugin
```

Table 19: Test coverage by module.

Module	Test File	Coverage
FixedPoint	<code>fixed_point.rs</code>	Arithmetic, fractions, display, deterministic multiply, checked division, lerp
RNG	<code>random.rs</code>	Deterministic sequences, domain separation, range bounds, chance function
Combat	<code>combat.rs</code>	Battle formula, casualty calculation, terrain bonuses, morale collapse
Cohesion	<code>cohesion.rs</code>	Decay toward equilibrium, component clamping, faction modifier application
Map	<code>map.rs</code>	Adjacency symmetry, no self-loops, pathfinding correctness, connectivity
Intrigue	<code>intrigue.rs</code>	Plot progress, secrecy decay, discovery, assassination success/failure
Economy	<code>economy.rs</code>	Tax collection, prosperity dynamics, population growth/decline
Plugin	<code>plugin.rs</code>	<code>on_init</code> , <code>on_tick</code> , <code>process_action</code> , <code>query_state</code> round-trips
Stress	<code>stress_tests.rs</code>	Multi-turn simulation, faction elimination, succession chains